

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ
Кафедра програмної інженерії

ЗВІТ

з лабораторної роботи № 4
«Code Review та рефакторинг»
з дисципліни «Основи програмної інженерії»

Студент: Волошко Данііл Сергійович

Група: ПЗПІ-25-4

Проект: Веб-додаток Style You

Харків — 2026

Тема та мета роботи

Тема: Code Review та рефакторинг.

Мета: Набути практичних навичок проведення взаємного огляду коду (Code Review), виявлення «запахів коду» (Code Smells), рефакторингу та застосування інструментів статичного аналізу (SonarLint, Code Metrics).

1. Результати Code Review одnogруппника

Буловедено детальний аналіз коду модулів системи в рамках Pull Request напарника. Виявлено наступні проблеми та надано рекомендації у вигляді коментарів до коду:

No	Рядок коду / Фрагмент	Проблема (Code Smell)	Категорія	Рекомендація
1	<pre>string role = "admin";</pre>	Незрозуміла назва змінної та Magic String	Dispensables	Перейменувати role та винести "admin" в enum UserRoles
2	<pre>if (user.Role == "Manager")</pre>	Магічний рядок (Magic String)	Maintainability	Використовувати перерахування (enum) замість рядкових літералів
3	<pre>void SendEmail(string a, string b, bool c, int d, string e)</pre>	Занадто багато параметрів (Long Parameter List)	Bloaters	Об'єднати параметри в клас EmailRequest
4	<pre>catch (Exception e) { }</pre>	Проковтування винятку (Empty Catch Block)	Error Handling	Додати <code>Logger.LogError(e)</code> , щоб виняток фіксувався
5	<pre>if (isValid == true)</pre>	Зайве порівняння з true	Bloaters	Спростити до: <pre>if (isValid)</pre>

2. Результати рефакторингу власного коду

На основі проведеного аналізу виконано рефакторинг власного модуля

```
StyleAdvisorService.cs
```

.

Операція 1: Усунення «магічних чисел» (Magic Numbers)

БУЛО:

```
if (bust > 250 || waist > 250 || hips > 250)
```

СТАЛО:

```
private const double MaxBodyMeasurement = 250.0;
if (bust > MaxBodyMeasurement || waist > MaxBodyMeasurement || hips >
    MaxBodyMeasurement)
```

Обґрунтування: Число 250 було захардкоджено тричі поспіль. Винесення його в іменовану константу підвищує читабельність та спрощує зміну порогового значення в майбутньому без пошуку по всьому файлу.

Операція 2: Спрощення логічного повернення (Simplify Boolean Return)

БУЛО:

```
if (age >= 18) return true;
return false;
```

СТАЛО:

```
return age >= 18;
```

Обґрунтування: Умовний оператор `if` є зайвим, оскільки логічний вираз вже повертає `bool` безпосередньо. Спрощений запис коротший, не вимагає галуження і легше сприймається під час читання.

Операція 3: Заміна імперативного циклу на декларативний (LINQ)

БУЛО:

```
int totalRequests = 0;
foreach (var count in dailyRequestCounts) {
    if (count < 0) throw new ArgumentException(...);
    totalRequests += count;
}
```

СТАЛО:

```
if (dailyRequestCounts.Any(c => c < 0))
    throw new ArgumentException(...);
int totalRequests = dailyRequestCounts.Sum();
```

Обґрунтування: Замість ручного циклу foreach використано методи розширення LINQ (.Any() та .Sum()). Код став декларативним, чітко виражає намір без деталей реалізації. Цикломатична складність методу знизилась.

3. Звіт регресійного тестування

Після виконання всіх трьох кроків рефакторингу було повторно запущено повний набір модульних тестів (NUnit). Усі 11 тестів успішно пройдені (100% Pass Rate). Регресійних помилок виявлено не було, функціональність системи збережена повністю.

4. Порівняння метрик «ДО / ПІСЛЯ»

Метрика	Значення ДО	Значення ПІСЛЯ	Аналіз змін
SonarLint Issues (Warnings)	5	0	Усі попередження статичного аналізатора усунені після рефакторингу.
Цикломатична складність	16 (загальна)	12 (загальна)	Зниження складності завдяки LINQ та прибиранню надлишкових умов.

5. Бонусне завдання (GitHub Copilot)

Під час виконання рефакторингу було задіяно ШІ-асистент (GitHub Copilot) з командою «Suggest refactoring» для методу обрахунку добових запитів. Асистент запропонував замінити цикл foreach методами .Any() та .Sum() з бібліотеки System.Linq. Пропозицію було перевірено, адаптовано до контексту задачі та інтегровано в код (Операція 3).

6. Підсумкова рефлексія

Найціннішим досвідом під час виконання лабораторної роботи стало усвідомлення різниці між «кодом, який працює» та «чистим кодом». Взаємний Code Review дає змогу поглянути на власний код очима стороннього розробника, що є критично важливим для виявлення «сліпих плям»: незрозумілих імен змінних, магічних рядків та зайвих логічних конструкцій. Практика рефакторингу наочно продемонструвала, що навіть невеликі цілеспрямовані зміни здатні суттєво підвищити підтримуваність (maintainability) програмного продукту та знизити когнітивне навантаження на команду.